

b)

- Análisis y diseño OO

¿Cuál es la Diferencia entre el Diagrama de Clases de Análisis y el Diagrama de Clases de Diseño? Realizar el Diagrama de Clases de Análisis del Ejercicio “a”.

El diagrama de análisis modela el problema del mundo real (el negocio) y el diagrama de diseño modela la solución técnica (el código y la arquitectura).

El diagrama de clases de análisis se construye en la etapa de análisis para comprender las reglas de negocio. Se enfoca solamente en el dominio del problema.

En estos diagramas solo aparecen las clases que se corresponden con objetos del mundo real (como Cliente, Producto) y los atributos se escriben en lenguaje natural, sin incluir el tipo de dato ni la visibilidad. Los métodos tampoco se colocan, se ponen funcionalidades muy generales (por ejemplo “calcular total”).

El diagrama de clases de diseño se elabora más avanzado el desarrollo, en la etapa de diseño. Además de las clases del diagrama de análisis se pueden incluir las clases técnicas (por ejemplo aquellas propias del MVC, o del patrón DAO). Los atributos se escriben junto con su tipo de dato y los símbolos de encapsulamiento. Además los métodos son precisos y se indica en ellos los parámetros y sus tipos.

- Patrones GRASP

Realizar una explicación de cada uno de los Patrones GRAPS. Además, investiga sobre los el Patrón Modelo Vista Controlador (MVC).

Los patrones GRASP (que significa General Responsibility Assignment Software Patterns) son reglas para asignar responsabilidades a las clases y objetos en el diseño Orientado a Objetos, es decir que responden a “¿qué clase debería llevar a cabo esto?”

Incluye 9 patrones:

1. Information expert: asigna una responsabilidad a la clase que tiene la información necesaria para cumplirla. Por ejemplo la clase Cliente no debería calcular el total a pagar de una factura porque es la clase Factura la que tiene una lista de los detalles con sus precios.
2. Creator: es para determinar qué clase debe ser la responsable de instanciar un nuevo objeto. Se le asigna a la clase A la responsabilidad de crear la clase B si: A contiene a

B, A usa de cerca a B, o A tiene los datos para inicializar a B. Siguiendo este patrón Factura debería encargarse de instanciar los objetos DetalleFactura.

3. Controller: dice que hay que asignar la responsabilidad de manejar los eventos de entrada del sistema (las interacciones con el usuario) a una clase que no sea la que se encarga de la interfaz de usuario. Por ejemplo, en la clase de la IU tenemos un botón "Agregar al carrito". Según el patrón, cuando el usuario haga click en el botón, otra clase (controladora) debe encargarse de añadir el producto al carro.
4. Bajo acoplamiento: con este patrón se busca diseñar cada clase de forma que sea lo más independiente posible del resto de clases.
5. Alta cohesión: con este patrón se busca diseñar las clases de forma tal que sus responsabilidades estén fuertemente relacionadas entre sí, en vez de que hagan un poco de todo.
6. Polimorfismo: consiste en asignar responsabilidades sobre un comportamiento que varía según el tipo a las clases (usando herencia o interfaces). Así, en vez de usar muchas estructuras condicionales se pueden crear interfaces o clases abstractas.
7. Pure fabrication: dice que deben crearse clases con el objetivo de que se cumplan la alta cohesión y bajo acoplamiento, aunque dichas clases no sean un concepto del mundo real (propias del dominio del negocio). Por ejemplo las clases que controlan el acceso a la base de datos en el patrón DAO.
8. Indirección: asignar la responsabilidad de mediar entre dos componentes a un objeto intermediario para evitar que se acoplen.
9. Protected variations: los puntos donde el sistema sea propenso a cambios deben envolverse en una interfaz para que dichos cambios afecten lo menos posible al resto del sistema.

Patrón MVC:

Es un patrón de arquitectura de software. Su objetivo es separar las responsabilidades de un programa en 3 componentes:

- **Modelo**: contiene las reglas y lógica del negocio. Se divide en capa de dominio, que tiene a las clases que modelan el negocio, capa de servicio que tiene las clases que se encargan de llevar a cabo toda operación con los objetos de la capa de dominio, incluso de instanciarlos. Por último está la capa de acceso a datos, que tiene a las clases repositorio que se encargan de la interacción con la base de datos.
- **Vista**: es la interfaz de usuario, su único trabajo es mostrar la información en pantalla y capturar interacciones.

- Controlador: es el intermediario entre la Vista y el Modelo. Toma la información capturada por la Vista y se la pasa al Modelo para que la procese, y con los resultados del Modelo actualiza la Vista.

Esta organización hace el código más limpio, fácil de mantener y de testear.

Explicar la vinculación que existe entre los patrones GRASP y el patrón MVC.

El MVC es un patrón de arquitectura de software que utiliza los patrones GRASP para definir los principios de dicha arquitectura.

Los patrones más evidentes son controller, ya que el concepto de clases controladoras es parte íntegra de la definición del MVC, el bajo acoplamiento y alta cohesión, la indirección y pure fabrication que son evidentes al implementar las clases de servicio.

Explicar los conceptos de FrontEnd y BackEnd. Explicar la arquitectura en Capas y como esta se vincula con el patrón MVC.

Un software se divide en dos partes:

- FrontEnd: es la interfaz de usuario. Su objetivo es mostrar la información por pantalla y permitir la interacción con el usuario.
- BackEnd: es la infraestructura del lado del servidor. Se encarga del procesamiento de los datos, garantizar la seguridad, conectarse a bases de datos y persistir la información.

“La arquitectura en capas es un modelo de diseño que organiza el software en niveles horizontales. Cada capa tiene una tarea única y se comunica solo con la capa de abajo. Esto ayuda a ordenar el código y facilita corregir errores.”

Este estilo arquitectónico no define en cuantas ni cuales capas se debe dividir el software, se centra en el concepto de división de responsabilidades.

El MVC es una implementación específica o especialización de la arquitectura en capas, que basándose en esta última si determina cuantas y en cuales capas estará dividida la aplicación.



Imagen de: <https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/capas>

- Spring: Framework Spring Boot.

¿Qué es Spring Boot? ¿Cuál es su Sitio Web Oficial? ¿Cómo se accede a la documentación?

Spring Boot es una herramienta del ecosistema de Spring (framework para desarrollo de aplicaciones empresariales en Java) para la creación rápida y de forma simple de aplicaciones grandes y robustas. Para esto se encarga de la configuración “de bajo nivel” (seguridad, manejo de transacciones, conexión a bases de datos, despliegue en servidores).

Sitio Web: <https://spring.io/projects/spring-boot>

Para acceder a documentación: en el sitio web hay una pestaña “Learn”. Ahí se indican las versiones del framework junto con sus Reference Docs.

¿Qué es Spring Tools? ¿Cuál es su Sitio Web Oficial?

“Herramientas de código abierto de última generación para Spring, compatibles con tu entorno de programación favorito. Ofrecen soporte de primer nivel para el desarrollo de aplicaciones empresariales basadas en Spring, independientemente de si prefieres Visual Studio Code, Cursor, Codespaces, Eclipse o Theia.”

(Extraído de la página oficial de Spring Tools)

Spring Tools es un conjunto de extensiones y complementos oficiales creados por el equipo desarrollador de Spring para facilitar el desarrollo usando Spring Boot en cualquier IDE. Por ejemplo autocompletado y análisis de código.

Sitio web: <https://spring.io/tools>

¿Qué es Spring Initializr? ¿Cuál es su Sitio Web Oficial?

Spring initializr es una herramienta oficial que sirve para generar la estructura básica de proyectos de Spring Boot de forma rápida y sencilla. Ofrece un servicio web y una API, que permite:

- Generación de código básica para Java, Kotlin y Groovy
- Soporte para .gitignore
- Varios “hook-points” para la generación de recursos personalizados
- Incluir dependencias

Sitio web oficial: <https://start.spring.io>

Repositorio de github: <https://github.com/spring-io/initializr>

¿Cuáles son las anotaciones en Spring Boot con las que se identifican una clase de tipo controlador, una clase de tipo servicio, una clase de tipo repositorio?

Explicar que implicancia tiene cada una, además de explicar cuál es el significado de una clase Controlador, una Clase de Servicio y una clase de Acceso a Datos.

Las clases controlador, de servicio y de persistencia tienen roles diferentes.

Controlador (mensajero): Se encarga de recibir peticiones, extraer datos y enviar estos a la clase de servicio; en sentido contrario, se encarga de devolver una respuesta al usuario.

- Anotaciones asociadas: @Controller o @RestController. Estas permiten que el componente maneje rutas y peticiones web. Al usar @RestController, Spring Boot habilita a la clase para escuchar peticiones HTTP y asume de forma automática que los datos devueltos deben ser serializados y convertidos a un formato como JSON antes de enviarse al cliente de regreso

Servicio: Aquí encontramos la lógica de negocio, es donde se aplican las reglas determinadas por el dominio de nuestro proyecto. Interactúa con la capa de persistencia cuando tiene que acceder a alguna información.

- Anotación asociada: `@Service`. Su implicancia principal es semántica, ya que sirve para que los desarrolladores puedan identificar fácilmente dónde se encuentran las reglas de negocio.

Persistencia: Es la clase encargada de comunicarse con la base de datos.

Anotación asociada: `@Repository`; esta anotación habilita la traducción automática de excepciones

Las tres anotaciones mencionadas son subtipos de la anotación `@Component`; usar cualquiera de estas le dice a Spring boot que debe generar una instancia de esa clase. Si ocurre un error nativo y específico del motor de base de datos, Spring lo intercepta de manera automática y lo convierte en una excepción genérica de Spring (nos permite independizarnos de la tecnología que estemos usando en la bbdd).

Explicar la Diferencia entre el Patrón DAO y Repository de Spring Boot.

En el patrón DAO, la comunicación con la bbdd se intenta ocultar de la capa de negocio: oculta las especificaciones del JDBC o de SQL del resto de la aplicación. Utiliza un lenguaje técnico donde sus métodos reflejan operaciones directas sobre los datos utilizando terminología de base de datos. Suele haber un DAO por tabla.

Mientras tanto el patrón repository de spring boot usa el diseño DDD (domain-driven design), que permite que la capa de negocio interactúe con la de repositorio como si estuviera consultando una lista o una estructura de datos mucho más simple que la bbdd. Enfoque centrado en el dominio: se asocia a las entidades en vez de a las tablas.

Conclusión: en el patrón DAO, buscamos los datos directamente de la bbdd, sin embargo, la lógica detrás de esto se encuentra oculta a la capa de negocios. En repository de Spring boot, tenemos una capa destinada a la comunicación con la bbdd que le provee toda la información que necesita a la capa de negocio en un formato más práctico (colecciones de objetos o entidades).

¿Para qué sirve el archivo pom.xml en un proyecto Spring Boot, que formato tiene y cuál puede ser su contenido?

El archivo pom.xml (Project Object Model) es el que almacena toda la información correspondiente al proyecto: las reglas de compilación, las dependencias externas y los complementos necesarios para ejecutar la aplicación. Solo se usa en proyectos que usan Maven, y le dice a este último cómo se debe empaquetar la aplicación.

Como es un archivo .xml, su formato se basa en etiquetas jerárquicas al estilo de html (<parent><parent/>, <dependencies><dependencies/>, etc.).

Etiquetas que nos solemos encontrar:

<parent>: Sirve para que el proyecto herede configuraciones por defecto y se encargue de gestionar las versiones compatibles de todas las librerías.

<groupId>: Describe la organización.

<artifactId>: Nombre específico del proyecto.

<properties>: Variables globales.

<dependencies>: Listado de librerías externas necesarias para poder ejecutar la aplicación.

Explicar el Patrón Inyección de Dependencia. ¿Cómo se aplica el patrón en Spring Boot?

En la programación orientada a objetos tradicional, si una clase principal necesita utilizar otra clase auxiliar para funcionar, habitualmente crea una instancia de esa dependencia por sí misma utilizando la palabra reservada "new".

El cambio que propone el patrón de inyección de dependencia, se basa en que estas herramientas, objetos auxiliares o dependencias deben ser proporcionados/inyectados desde el exterior. Esto permite que las clases tengan un mayor grado de independencia (que resulta en mayor mantenibilidad y reusabilidad).

En spring boot, la aplicación del patrón consiste en dos fases:

1. Creación de los beans: Al levantar nuestra aplicación, el framework detecta anotaciones de controlador, servicio y persistencia y crea objetos de estas clases que luego guardará en un espacio de memoria destinado al "contexto de la aplicación". A cada uno de estos objetos instanciados, guardados y administrados exclusivamente por el framework se le denomina "Bean".
2. Inyección: Ocurre al momento de ensamblar los beans. Si una clase requiere a otra para funcionar, debemos usar la anotación @Autowired. Cuando spring boot va a instanciar la clase y lee esa anotación, detiene el proceso y busca en su "contexto de aplicación" un bean cuyo tipo de dato coincida con el solicitado. Si encuentra uno, toma la referencia en memoria a ese Bean y se lo pasa a la clase que lo solicitó.

- Diagrama de Secuencia:

¿Qué es el Diagrama de Secuencia y para qué sirve? Describir los elementos gráficos que utiliza el diagrama de Secuencia. ¿Cuál es la diferencia entre el Diagrama de Secuencia de Análisis y el Diagrama de Secuencia de Diseño? Explicar un ejemplo de ABM de una entidad Nacionalidad que posee el atributo nombre, cómo se representan en un Diagrama de Secuencia de Diseño y como son los mensajes entre la clase Controladora, la clase de Servicio y la clase de Acceso a Datos.

Un Diagrama de Secuencia es un diagrama de interacción del lenguaje UML que muestra cómo los objetos o componentes de un sistema se comunican entre sí a lo largo del tiempo para llevar a cabo un proceso. Su propósito principal es modelar el comportamiento dinámico del sistema, revelando el orden cronológico de los mensajes y ayudando a los desarrolladores a identificar la lógica de negocio antes de programar. Se lee de arriba hacia abajo de izquierda a derecha.

Los elementos gráficos que existen en un diagrama de secuencia son:

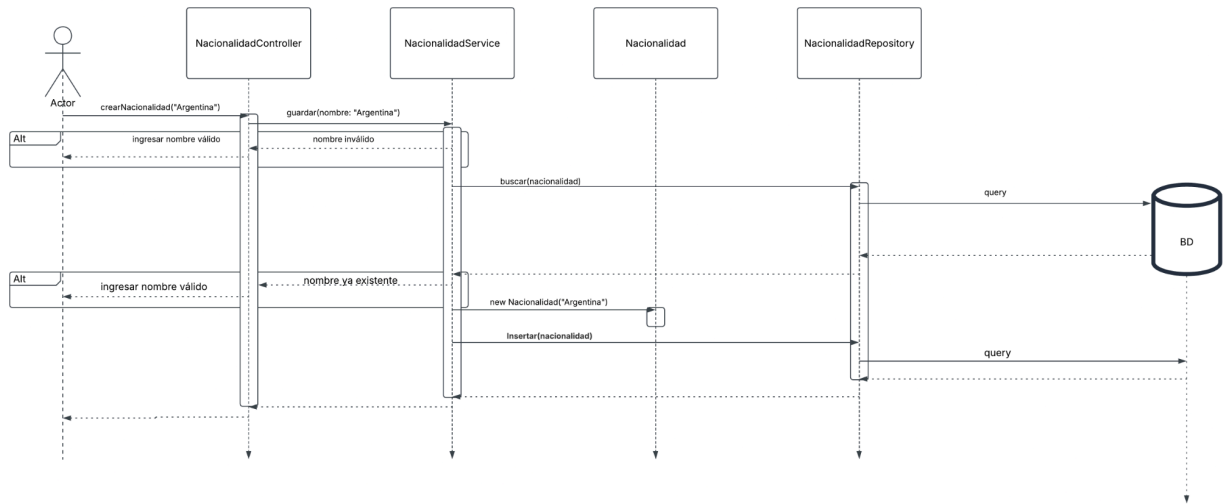
- **Actor:** Representa a un usuario o sistema externo que inicia la interacción. Se dibuja típicamente como un muñeco de palo.
- **Objeto:** Los participantes del sistema. Se representan con un rectángulo en la parte superior que contiene el nombre del objeto o la clase.
- **Línea de vida:** Una línea vertical punteada que desciende desde el objeto. Representa la existencia del objeto durante el tiempo que dura la interacción.
- **Activación:** Un rectángulo vertical superpuesto sobre la línea de vida. Indica el período exacto en el que el objeto está activo y ejecutando un proceso.
- **Mensajes:** Representan la comunicación.
 - *Síncrono:* Flecha continua con punta sólida. El emisor espera una respuesta antes de continuar.
 - *Asíncrono:* Flecha continua con punta abierta. El emisor no espera respuesta y continúa su ejecución.
 - *Respuesta (Return):* Flecha punteada con punta abierta. Indica el retorno de datos o el fin del método.
 - *Creación:* Apunta directamente a la caja de un nuevo objeto para indicar que está siendo instanciado.
- **Fragmentos:** Cajas que encierran una parte del diagrama para representar lógica compleja, como bucles, alternativas/condicionales, o componentes opcionales.

Diferencias entre el diagrama de análisis y el de diseño:

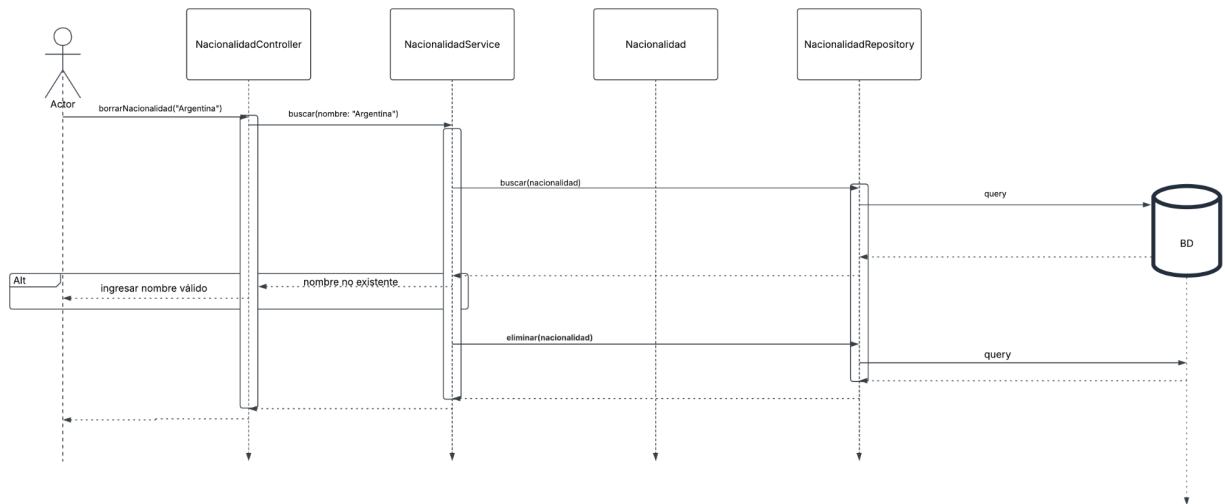
Característica	Diagrama de Análisis	Diagrama de Diseño
Enfoque	Conceptual. Qué hace el sistema.	Técnico. Cómo se implementa el sistema.

Componentes	Clases básicas sin dividir las en patrón MVC.	Clases de la arquitectura real de software (Controladores, Servicios, Repositorios/DAO, BD).
Nivel de detalle	Alto nivel. Los mensajes son acciones descriptivas (ej. "Validar datos").	Bajo nivel. Los mensajes son métodos reales con firmas, parámetros y tipos (<code>boolean validarDatos(String)</code>).
Dependencia Tecnológica	Agnóstico. No asume lenguajes de programación, frameworks ni bases de datos.	Específico. Refleja las convenciones del lenguaje, inyección de dependencias o llamadas a base de datos.

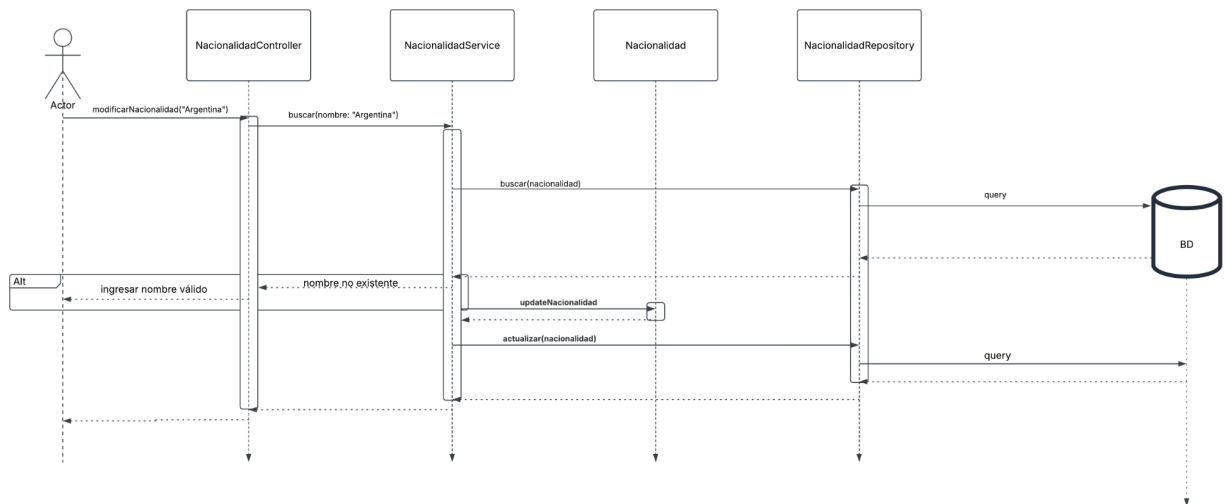
Alta



Baja



Modificación



- Diagrama de Paquetes:

¿Qué es el Diagrama de Paquetes y para qué sirve? Describir los elementos gráficos que utiliza el diagrama de Paquetes.

El diagrama de paquetes es uno de los diagramas estructurales comprendidos en UML. Es utilizado para definir los distintos paquetes a nivel lógico que forman parte de la aplicación y la dependencia entre ellos. Sirve para gestionar la complejidad al dividir sistemas masivos y complejos en subsistemas (paquetes) más pequeños y fáciles de entender, visualizar la arquitectura al proporcionar una vista de alto nivel de la estructura general del software, mostrar dependencias ya que permite identificar qué módulos del sistema dependen de otros y también controlar el acceso permitiendo definir “namespaces” y definir la visibilidad, determinando qué paquetes pueden estar expuestos a otros y cuáles se ocultan.

Elementos gráficos que lo componen:

- **Paquete:** Es el elemento central y se representa gráficamente como una carpeta con una pestaña en la esquina superior izquierda. Si el paquete se dibuja cerrado, su nombre va en el centro del rectángulo principal. Si está abierto para mostrar su contenido, el nombre va en la pestaña superior.
- **Dependencia:** Se dibuja como una flecha de línea discontinua con la punta abierta. Señala desde el paquete que necesita la información (cliente) hacia el paquete que la provee. Indica que los cambios en el proveedor pueden afectar al cliente.

- **Elementos Internos:** Son los componentes que viven dentro del paquete. Pueden ser clases, interfaces, casos de uso, e incluso otros paquetes.
 - **Estereotipos:** Es texto encerrado entre comillas angulares (`<<ejemplo>>`). Se usan frecuentemente sobre las flechas de dependencia para especificar cómo se relacionan los paquetes, como `<<import>>` (el paquete cliente tiene acceso público al contenido del proveedor) o `<<access>>` (acceso privado).
 - **Visibilidad:** Se indica con símbolos matemáticos antes del nombre del elemento interno para controlar su exposición. Los más comunes son **+** (público, accesible desde afuera), **-** (privado, solo accesible dentro del paquete) y **#** (protegido).
- FrontEnd: Thymeleaf.

Identificar las principales características de Thymeleaf

- **Plantillas naturales:** A diferencia de otros motores, los archivos de Thymeleaf son HTML válido. Esto significa que **puedes abrir una plantilla de Thymeleaf directamente en un navegador web** y verás su estructura y diseño básico, incluso si la aplicación Java no está en ejecución. Los diseñadores web y los desarrolladores backend pueden trabajar sobre el mismo archivo sin que el código de la plantilla rompa el diseño estático.
- **Sintaxis basada en atributos:** La lógica dinámica no se inserta rompiendo las etiquetas HTML, sino que se añade como atributos dentro de las mismas etiquetas. **Ejemplo:** En lugar de usar etiquetas especiales para un texto dinámico, Thymeleaf utiliza atributos con el prefijo **th**: (por ejemplo, `<p th:text="{mensaje}">Texto por defecto</p>`). Si el motor lo procesa, reemplazará "Texto por defecto" por la variable; si lo abres como archivo estático, verás "Texto por defecto".
- **Integración con Spring:** Ofrece una integración profunda y "fuera de la caja" (out-of-the-box) con **Spring MVC** y **Spring Boot**, facilitando el manejo de formularios, validaciones, internacionalización (la capacidad de que tu aplicación se adapte a **diferentes idiomas y regiones** de forma automática) y vinculación de datos (Ejemplo: un usuario ingresa sus datos para registrarse y Thymeleaf con Spring empaquetan esos datos para crear el objeto Usuario directamente).

¿Cuál es su sitio web Oficial? ¿Cómo se accede a la documentación oficial?

El sitio web oficial de Thymeleaf es: <https://www.thymeleaf.org/index.html> y para acceder a su documentación oficial el sitio es el siguiente: <https://www.thymeleaf.org/documentation.html>

¿Cómo se vincula Thymeleaf con HTML?

Para vincular a Thymeleaf con tu archivo HTML se tiene que agregar `<xmlns:th="http://www.thymeleaf.org">` a tu etiqueta `<html>`, por ejemplo:

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Mi página</title>
</head>
<body>
    <!-- acá adentro ya se puede usar Thymeleaf -->
</body>
</html>
```

Luego los atributos de Thymeleaf se inyectan dentro de las etiquetas HTML, por ejemplo `<p th:text="{nombreUsuario}">Juan Pérez</p>`, `<a th:href="@{/perfil}">Ir al perfil</a>`, etc.

El intermediario para que HTML y Java puedan conectarse se llama Modelo donde éste último funciona como una caja donde Java va a guardar la información que buscó en la base de datos pero etiquetándolos, luego Java le pasa el Modelo al archivo HTML, Thymeleaf escanea al HTML buscando atributos “th” y por cada vez que encuentra uno busca en el Modelo su valor que Java le asignó para inyectarlo en el HTML.